```
#include <iostream>
#include <iomanip>
using namespace std;

constexpr long long MIN_BALANCE_PAISE = 5000;

// ========================================
// PROCEDURAL VERSION
// ========================================

long long proceduralBalance = 10000;

bool proceduralCanTravel(long long balance, long long fare) {
    return fare >= 0 &&
           balance >= fare &&
           balance - fare >= MIN_BALANCE_PAISE;
}

bool proceduralDeduct(long long& balance, long long fare) {
    if (!proceduralCanTravel(balance, fare)) {
        return false;
    }

    balance -= fare;
    return true;
}

void proceduralJourney(long long fare) {
    if (proceduralDeduct(proceduralBalance, fare)) {
        cout << "Procedural: Journey successful. ";
```

```
        cout << "Balance Rs "
             << proceduralBalance / 100 << '\n';
    } else {
        cout << "Procedural: Journey denied. ";
        cout << "Balance Rs "
             << proceduralBalance / 100 << '\n';
    }
}

// ========================================
// OBJECT-ORIENTED VERSION
// ========================================

class SmartCard {
private:
    long long balancePaise;
    static constexpr long long MIN_BALANCE = 5000;

public:
    SmartCard(long long initialBalance)
        : balancePaise(initialBalance) {}

    long long getBalance() const {
        return balancePaise;
    }

    bool canTravel(long long fare) const {
        return fare >= 0 &&
               balancePaise >= fare &&
               balancePaise - fare >= MIN_BALANCE;
```

```
    }

    bool deduct(long long fare) {
        if (!canTravel(fare)) {
            return false;
        }

        balancePaise -= fare;
        return true;
    }
};

void objectOrientedJourney(SmartCard& card, long long fare) {
    if (card.deduct(fare)) {
        cout << "OOP: Journey successful. ";
        cout << "Balance Rs "
             << card.getBalance() / 100 << '\n';
    } else {
        cout << "OOP: Journey denied. ";
        cout << "Balance Rs "
             << card.getBalance() / 100 << '\n';
    }
}

int main() {
    cout << "NAMMA METRO FARE SYSTEM\n\n";

    cout << "PROCEDURAL VERSION\n";
    proceduralJourney(1000);  // Rs 10
    proceduralJourney(3600);  // Rs 36
```

```
    proceduralJourney(1000);  // Rs 10

    cout << "\nOBJECT-ORIENTED VERSION\n";

    SmartCard card(10000);    // Rs 100

    objectOrientedJourney(card, 1000);
    objectOrientedJourney(card, 3600);
    objectOrientedJourney(card, 1000);

    return 0;
}
```